TwinCAT Boot + HMI Server update scripts (Linux)

Linux version of the Windows scripts. It works the same way and uses the same package workflow, but is written in bash for TwinCAT on Linux (Beckhoff RT Linux / Debian, systemd).

Where	Run	What it does
Bench system	`Create-UpdatePackage.sh`	Uses ADS to switch TwinCAT to Config mode and stops the HMI Server (`TcHmiSrv`). Copies the Boot folder and HMI Server folder to `<USB>/TwinCAT_Update` with SHA-256 checksums, then restarts both.
Machine IPC	`Apply-Update.sh` (started by the PLC)	Finds and verifies the package on the USB stick. Uses ADS to switch to Config mode and stops the HMI Server. Backs up the old version, replaces the folders, verifies them, rolls back if anything fails, then reboots.

```
Apply-Update.sh / Create-UpdatePackage.sh   launchers
Config.sh                                   paths, service name, ADS tool, options  <-- check this
first
scripts/Common.sh                           shared functions
scripts/New-UpdatePackage.sh                bench logic
scripts/Install-UpdatePackage.sh            machine logic
```

Requirements

- bash 4.3+, GNU coreutils / findutils, util-linux (`lsblk`, `flock`, `mount`) and systemd. All of these come with Debian.
- `rsync`. If it's missing, install it: `sudo apt install rsync`.
- An ADS command-line tool to switch TwinCAT between Run and Config mode. The default is Beckhoff's `adstool` (see setup step 2).
- Root rights. The launchers run `sudo` themselves when started by a normal user.

One-time setup

1. **Check the paths in `Config.sh`** against a real IPC. The first folder that exists is used, and the log shows which one. The defaults are a starting point, so **confirm them on your image**:

   - Boot: `/etc/TwinCAT/3.1/Boot` (RT Linux) or `/usr/local/etc/TwinCAT/3.1/Boot` (TwinCAT/BSD layout)
   - HMI: `/var/lib/TcHmiSrv` or `/etc/TwinCAT/Functions/TF2000-HMI-Server`

   If no folder exists for a required item, the scripts stop before changing anything. The error lists the paths they checked.

2. **Check ADS works.** The scripts read and set the TwinCAT system state with the command in `ADS_TOOL` (default `adstool 127.0.0.1 --port=10000`). They append `state` to read it and `state <n>` to set it. Test it by hand:

   ```
   adstool 127.0.0.1 --port=10000 state      # should print 5 (Run) or 15 (Config)
   ```

If your image uses a different tool or target address, change `ADS_TOOL`. On the machine, ADS is checked before anything is stopped. If it doesn't answer, the update stops and nothing is changed.

3. **Check the version packages.** `XAR_PACKAGES` and `HMI_PACKAGES` name the installed packages whose versions must match between bench and machine. List the real names with:

   ```
   dpkg -l | grep -i -e tc31 -e tf2000
   ```

4. **Check the HMI service name** (`HMI_SERVICE_NAME`, default `TcHmiSrv`): `systemctl status TcHmiSrv`.

5. **On each machine**, copy this whole folder to a fixed local path, for example `/opt/TwinCAT_UpdateTool/`. The USB stick only needs to hold the `TwinCAT_Update` package folder.

Headless IPCs usually don't mount USB sticks automatically. With `AUTO_MOUNT_USB=1` (the default), the scripts mount any unmounted USB partition under `/run/twincat-update/`, and flush and unmount it again at the end. A stick that was already mounted is left mounted.

PLC trigger (Tc2_Utilities `NT_StartProcess`)

```
fbStartUpdate(
    NETID     := '',
    PATHSTR   := '/bin/bash',
    DIRNAME   := '/opt/TwinCAT_UpdateTool',
    COMNDLINE := '/opt/TwinCAT_UpdateTool/Apply-Update.sh',
    START     := bHmiUpdateButton,   // rising edge starts it
    TMOUT     := T#5S);
```

The script needs root. If the TwinCAT runtime does not start processes as root on your image, add a `sudoers` rule for this one script instead of running it with a password prompt, e.g. `/etc/sudoers.d/twincat-update`:

```
<runtime user> ALL=(root) NOPASSWD: /bin/bash /opt/TwinCAT_UpdateTool/Apply-Update.sh, /bin/bash
/opt/TwinCAT_UpdateTool/scripts/Install-UpdatePackage.sh
```

TwinCAT keeps running until the package has been fully validated. If the stick is missing or the package is bad, nothing is stopped and the machine keeps running.

Optional arguments

`Apply-Update.sh` (machine)

Argument	What it does
`--allow-version-mismatch`	Applies the package even if the XAR / TF2000 versions differ from this machine. The mismatch is still logged. Manual use only.
`--no-reboot`	Doesn't reboot at the end. Restarts the HMI Server and puts TwinCAT back into Run mode instead.
`--package-path <folder>`	Uses this package folder instead of searching the USB drives, `/media`, `/run/media` and `/mnt`. Needed if more than one drive holds a package.
`--config <file>`	Uses a different config file instead of `Config.sh`.

`Create-UpdatePackage.sh` **(bench)**

Argument	What it does
`--destination <folder>`	Where to build the package, e.g. `/media/usb`. Default: the USB drive the script runs from, or the only USB drive present.
`--no-stop`	Copies without stopping TwinCAT / the HMI Server first. Not recommended: files can change mid-copy.
`--leave-stopped`	Leaves TwinCAT in Config mode and the HMI Server stopped afterwards.
`--config <file>`	Uses a different config file instead of `Config.sh`.

Set `NOPAUSE=1` to stop `Create-UpdatePackage.sh` waiting for Enter at the end.

Result and logs (machine)

- `/var/lib/TwinCAT_UpdateBackups/last_update_status.txt`: `SUCCESS` / `FAILED - nothing changed` / `FAILED - previous version restored` / `FAILED - ROLLBACK FAILED`
- `/var/lib/TwinCAT_UpdateBackups/Logs/Apply_<time>.log`. A copy also goes to `<USB>/TwinCAT_Update/Logs/`.
- `/var/lib/TwinCAT_UpdateBackups/<time>_before_update/`: the old Boot and HMI folders. The last 5 are kept (`BACKUPS_TO_KEEP`).

Exit codes: 0 ok, 1 failed (nothing changed), 2 failed and rolled back, 3 rollback failed (the IPC is left in Config mode and **not** rebooted), 4 already running.

The reboot is scheduled as a systemd timer, so it happens even if the script is stopped during the countdown. To cancel it: `systemctl stop twincat-update-reboot.timer`.

Manual rollback to an old version: copy `<time>_before_update/Boot` and `/HMI` back to their folders while TwinCAT is in Config mode and `TcHmiSrv` is stopped, then reboot. Or put that folder on a stick as `TwinCAT_Update`, change `Purpose=PreUpdateBackup` to `Purpose=UpdatePackage` in its `manifest.txt`, and run Apply-Update. Each backup has its own `versions.txt`, so the version check still applies.

Package format

```
TwinCAT_Update/
  Boot/  HMI/          the copied folders
  Boot.sha256          checksums (sha256sum format: "sha256sum -c Boot.sha256" from inside Boot/
checks it by hand)
  HMI.sha256
  versions.txt         XAR / TF2000 versions of the bench
  manifest.txt         written last; its presence marks the package as complete
  create.log           bench log
  Logs/                logs copied back from the machines
```

Packages are not interchangeable between the Windows and Linux versions (different paths and manifest format).

Important behaviour

- **PLC persistent data is kept.** `PRESERVE_PERSISTENT_DATA=1` means `*.bootdata` files are neither packaged nor overwritten, so each machine keeps its own retained values. If the new PLC software changes persistent variables, check how TwinCAT handles that before you rely on it.
- **Mirror copy.** The target folders end up exactly like the package. Stale files are removed, except excluded files, which are left alone.
- **File ownership.** USB sticks (FAT/exFAT) can't store Linux owners or permissions, so installed files get the owner and group of the target folder (e.g. the owner of `/etc/TwinCAT/3.1/Boot`). Existing files keep their permissions. Symlinks in the source folders are not packaged. The local backups are exact copies (owners, permissions, symlinks), so a rollback restores everything exactly.
- **Version check (exact match).** The bench records the installed version of the XAR and TF2000 packages (e.g. `3.1.4026.12`, the Debian revision is dropped) in `versions.txt`. Apply-Update reads the same values on the machine. If either differs, or can't be read, it stops with an error **before anything is stopped or changed**. The bench refuses to build a package if it can't read either version.
- **Hardware binding.** The Boot folder holds the bench's I/O configuration. EtherCAT adapters are bound to a MAC address or PCI slot, so the bench and machines should be the same IPC model and use the same adapter binding. Licences are not part of the Boot folder and don't move with it.
- Incomplete packages are written as `TwinCAT_Update.partial` and are never picked up. The previous package on the stick is kept as `TwinCAT_Update_previous`. The stick is flushed (`sync`) before the bench reports success.